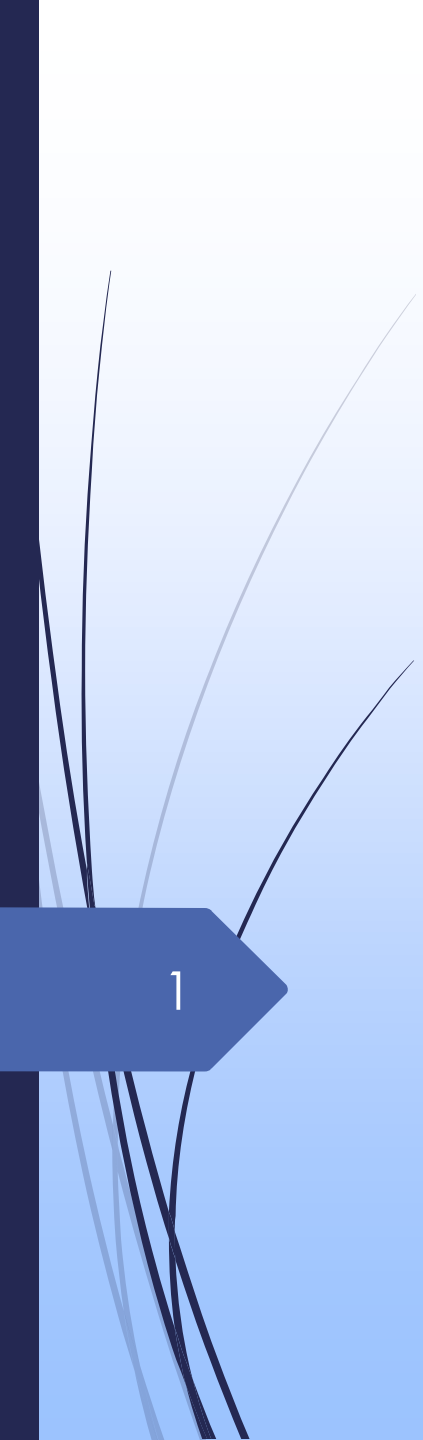




EECS 280 – Lecture 2

Functions, Types, Control Structures

Functions

- Functions allow for **procedural abstraction**.
 - e.g. Use `sqrt()` to compute a square root
 - e.g. call `extract_columns()` on P1 to read data from a file
- The **interface** for a function describes how we use it:
 - e.g. `sqrt()`: "Pass in a number. It returns the square root."
- The **implementation** is the actual code for the function.

Function Definitions

Parameter
Declaration List

Return Type

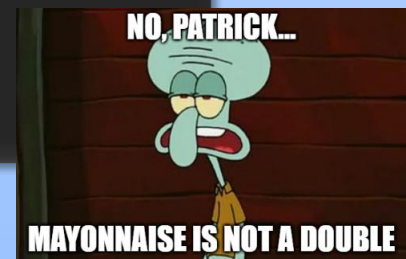
```
double rectangle_area(double w, double h) {  
    return w * h;  
}
```

Function Body:
Implementation code
goes between the { }

- Parameters and returns are statically type checked.

```
double rectangle_area(double w, double h) {  
    return "hello"; // ERROR  
}
```

```
int main() {  
    string s = rectangle_area(3, 4); // ERROR  
    rectangle_area(3, "mayonnaise"); // ERROR  
}
```



Function Examples

```
double abs(double x) {  
    if (x < 0) {  
        return -x;  
    } else {  
        return x;  
    }  
}
```

A function may contain multiple paths to a return.

```
void print(string message, int num_times) {  
    for (int i = 0; i < num_times; i++) {  
        cout << message << endl;  
        cout << x; // ERROR - no x in this scope  
    }  
    return;  
}
```

A void function doesn't return anything.

May optionally omit this return;

```
int main() {  
    cout << abs(-3) << endl;  
    string x = "hello";  
    print(x, 3);  
}
```

Special case: main() has a default return 0;

Function Scope

Separate functions have separate local scopes.

You can use the same name for different variables in different scopes.

C++ Standard Library Types

- Generally considered part of C++, but must be brought in via a proper `#include` directive.
- `using namespace std;` allows you to omit the `std::`

Type	Description	Example
<code>std::string</code>	A "string" of characters	<pre>#include <string> string s = "hello there";</pre>
<code>std::vector</code>	Generic sequential container	<pre>#include <vector> vector<int> v = {1, 2, 3};</pre>
<code>std::size_t</code>	Unsigned integer for sizes or indices of containers	<pre>#include <vector> for (size_t i = 0; i < v.size(); ++i) { cout << v[i] << endl; }</pre>
<code>std::pair</code>	Pair of two typed values	<pre>#include <utility> pair<int, bool> = {142, false};</pre>
...	...	...

std::string

- ▶ A std::string represents text, a sequence of char

```
#include <string>
using namespace std;
string str1 = "make a ";
string str2 = "wish";

str1 < str2; // true, alphabetic comparison

cout << str1.size() << endl; // 7 (includes spaces)

string str3 = str1 + str2; // "make a wish"
str3[0] = str3[7];
str3[7] = 'f';
cout << str3 << endl; "wake a fish"
```

We'll come back to strings and program I/O later in the course...

std::pair

- ▶ std::pair represents an ordered pair of values.
- ▶ A **template parameter** specifies the element type.
- ▶ For example:

```
std::pair<int, bool> p1;  
p1.first = 5;  
p1.second = false;  
  
std::pair<string, int> p2;  
p2.first = "hello";  
p2.second = 4;
```

std::vector

- ▶ A `std::vector` is a homogenous, sequential container.
- ▶ A **template parameter** specifies the element type.

```
#include <vector>
using namespace std;
vector<double> nums = {1, 5, 3.5, 6.5};
vector<string> pets = {"cat", "dog", "fish"};
```

- ▶ There are several different ways to initialize a vector.*

```
vector<int> v; // empty
vector<int> v(5); // 0, 0, 0, 0, 0
vector<int> v(3, 42); // 42, 42, 42
vector<int> v = {3, 5, 42, 28};
```

*These initializations each use different **constructors** for the vector class.

vector Indexing

- Use the [] operator:

```
vector<int> v = {3, 5, 42, 28};  
cout << v[0] << endl;  
v[3] = 7;  
  
v[4] = 100; // Out of bounds... undefined behavior!
```

- Use the .at() function:

```
vector<int> v = {3, 5, 42, 28};  
cout << v.at(0) << endl;  
v.at(3) = 7;  
  
v.at(4) = 100; // Out of bounds.. throws an exception
```

*The [] operator is slightly faster since it doesn't check the provided index.

.push_back() and .pop_back()

- Vectors are dynamically sized containers:

```
vector<int> v;  
// v contains {}  
  
v.push_back(1);  
v.push_back(2);  
v.push_back(3);  
// v contains {1, 2, 3}  
  
v.pop_back();  
// v contains {1, 2}  
  
v.clear();  
// v contains {}
```

Vector functions

- Many different functions can be called on a vector, using the dot notation. Here are a few common ones:

<code>size</code>	Returns the number of elements.
<code>front</code>	Returns a reference to the first element.
<code>back</code>	Returns a reference to the last element.
<code>at</code>	Works like indexing, but does bounds checking.
<code>empty</code>	Returns whether the vector is empty (as a bool).
<code>clear</code>	Removes all elements from the vector.
<code>push_back</code>	Adds a new element to the back of the vector.
<code>pop_back</code>	Removes the last element in the vector.

Iteration with **for** loops

- ▶ A sample loop to print out "0 a 1 a 2 a 3 a 4 a "

```
int main() {  
    for (int x = 0; x < 5; x = x + 1) {  
        cout << x << " ";  
        cout << "a ";  
    }  
    cout << "done!" << endl;  
}
```

May also be written:

```
x += 1;  
++x;  
x++;
```

```
int main() {  
    for ( 1 ; 2 ; 4 ) {  
        3  
    }  
}
```

1. Initialize – ONLY ONCE

Repeat:

2. Check Condition
3. Loop Body
4. Increment Step

Exercise: Sum of Vector

- Fill in the code to compute the sum of the vector.

```
int main() {  
    vector<double> v = {1, 5, 3.5, 6.5};  
  
    // Declare accumulator variable to hold the sum  
    _____  
  
    // Traverse by index from 0 ... v.size()-1  
    for (_____; _____; _____) {  
        // Access each element by index and add to sum  
        _____  
    }  
  
    cout << "Sum: " << sum << endl;  
}
```

Exercise: Sum of Vector

- ▶ This code compiles yet still contains 3 bugs. Find them.

Should be double.

```
int main() {  
    vector<double> v = {1, 5, 3.5, 6.5};  
  
    // Declare accumulator variable to hold the sum  
    int sum = 0;  
  
    // Traverse by index from 0 ... v.size()-1  
    for (int i; i <= v.size(); ++i) {  
  
        // Access each element by index and add to sum  
        sum += v[i];  
    }  
  
    cout << "Sum: " << sum << endl;  
}
```

Must initialize to 0.

Should be strict <

Solution: Sum of Vector

- Compute the sum of a vector.

```
int main() {  
    vector<double> v = {1, 5, 3.5, 6.5};  
  
    // Accumulator variable to compute the sum  
    double sum = 0;  
  
    // Traverse by index from 0 ... v.size()-1  
    for (int i = 0; i < v.size(); ++i) {  
        // Access each element by index and add to sum  
        sum += v[i];  
    }  
  
    cout << "Sum: " << sum << endl;  
}
```

Iteration with **while** loops

➤ A while loop is a simplification of a for loop.

```
int main() {  
    for ( 1 ; 2 ; 4 ) {  
        3  
    }  
}
```

1. Initialize – ONLY ONCE

Repeat:

2. Check Condition
3. Loop Body
4. Increment Step

```
int main() {  
    while ( 2 ) {  
        3  
    }  
}
```

Repeat:

2. Check Condition
3. Loop Body

size_t

- ▶ The vector `.size()` function returns a `size_t`:
 - ▶ An unsigned integer type for sizes or indices of containers

```
for (int i = 0; i < v.size(); ++i) {
```

Some compilers may give a warning here about the comparison between *signed* and *unsigned* ints.

- ▶ Fix: Use `size_t` for indexing.

```
for (size_t i = 0; i < v.size(); ++i) {
```

- ▶ Fix: Put in an explicit conversion.

```
for (int i = 0; i < static_cast<int>(v.size()); ++i) {
```

if / else

```
if ( condition ) {
```

run if true

```
}
```

```
else {
```

run if false

```
}
```

condition

Must be a bool (or a type that can convert to bool).

```
int absolute_value(int x) {  
    if (x < 0) {  
        cout << "inverting" << x << endl;  
        x = -x;  
    }  
    return x;  
}
```

Note: use remainder mod 2 to check if even or odd.

```
void describe_parity(int x) {  
    if (x % 2 == 0) {  
        cout << x << " is even" << endl;  
    } else {  
        cout << x << " is odd" << endl;  
    }  
}  
  
void describe_sign(int x) {  
    if (x > 0) {  
        cout << x << " is positive" << endl;  
    }  
    else if (x < 0) {  
        cout << x << " is negative" << endl;  
    }  
    else {  
        cout << x << " is zero" << endl;  
    }  
}
```

Braces

- The curly braces are technically not required on an if/else if you only have a single statement.
- However, it's easy to introduce bugs if they're omitted.
- **Recommendation**
Always use braces.

```
int absolute_value(int x) {  
    if (x < 0)  
        cout << "inverting" << x << endl;  
        x = -x;  
    return x;  
}
```

Bug! `x = -x;` is not within the if. Indentation doesn't matter, the `{}` do.

```
void describe_parity(int x) {  
    if (x % 2 == 0)  
        cout << x << " is even" << endl;  
    else  
        cout << x << " is odd" << endl;  
}  
  
void describe_sign(int x) {  
    if (x > 0)  
        cout << x << " is positive" << endl;  
  
    else if (x < 0)  
        cout << x << " is negative" << endl;  
  
    else  
        cout << x << " is zero" << endl;  
}
```

Exercise: Minimum of Vector

- Fill in the code to find the minimum value in the vector.

```
int main() {  
    vector<double> v = {1, 5, 3.5, 6.5};  
  
    // Keep track of the "best" candidate we've seen.  
    _____;  
  
    for (size_t i = 0; i < v.size(); ++i) {  
        // If v[i] is less than the current min, update min.  
        _____  
    }  
  
    cout << "Min: " << min << endl;  
}
```

Exercise: Minimum of Vector

- ▶ This code compiles yet still contains 2 bugs. Find them.

```
int main() {  
    vector<double> v = {1, 5, 3.5, 6.5};  
  
    // Keep track of the "best" candidate we've seen.  
  
    double min = 0;  Initializing to 0 may cause issues.  
  
    for (size_t i = 0; i < v.size(); ++i) {  
        // If v[i] is less than the current min, update min.  
        if(v[i] < min) {  
            double min = v[i];  
        }  
    }  This declares a separate min variable in the nested scope, rather than updating the original.  
  
    cout << "Min: " << min << endl;  
}
```

Solution: Minimum of Vector

- Fill in the code to find the minimum value in the vector.

```
int main() {  
    vector<double> v = {1, 5, 3.5, 6.5};  
  
    // Keep track of the "best" candidate we've seen.  
    // Assume for now it's just the first element.  
    double min = v[0];  
  
    for (size_t i = 0; i < v.size(); ++i) {  
        // If v[i] is less than the current min, update min.  
        if(v[i] < min) {  
            min = v[i];  
        }  
    }  
  
    cout << "Min: " << min << endl;  
}
```

Logical Operations

- C++ supports most common logical operations.

	Operator	Alternative Operator	Example	Result
Logical And	&&	and	<code>2 < 3 && 5 > 6</code>	<code>false</code>
Logical Or	 	or	<code>2 < 3 5 > 6</code>	<code>true</code>
Exclusive Or	None			
Not	!	not	<code>!('a' == 'b')</code>	<code>true</code>

- C++ also includes “bitwise operators”, `&`, `|`, `!`, and `^`.
 - These manipulate the binary representation of data.

Short-Circuit Evaluation

- ▶ `&&` and `||` have **short-circuit** evaluation:
 1. Evaluate the left operand first.
 2. Evaluate the right operand ONLY if needed.
- ▶ Common Pattern:
Use the left operand to establish the safety of the right.

```
int main() {  
    vector<double> v; // Assume v contains some data  
  
    // Print elements until the first 0  
    for (size_t i = 0; _____; ++i) {  
        cout << v[i] << endl;  
    }  
}
```

Minute Exercise

A `i < v.size() && v[i] != 0`

B `v[i] != 0 && i < v.size()`

break; and continue;

- A break; statement exits the containing loop.

```
// Print elements until the first 0
for (size_t i = 0; i < v.size(); ++i) {
    if (v[i] == 0) {
        break;
    }
    cout << v[i] << endl;
}
```

- A continue; statement skips the rest of the loop body.

```
// Print the square root of non-negative numbers
for (size_t i = 0; i < v.size(); ++i) {
    if (v[i] < 0) {
        continue;
    }
    cout << sqrt(v[i]) << endl;
}
```

We'll start again in five minutes.



Participation

bit.ly/469o3DR

